

מטלה 4

מגישים: עמית ביטון, דוד קיטינברג, שירה בן ארצי, אלדין אבו חגלה
ראש צוות: עמית ביטון
ת"ז: 213763519, 322315300, 211520127, 322231580

הקדמה

נתחיל בהסבר קצר על מבנה הפרויקט. בפרויקט שלנו בחרנו בארכיטקטורת **MVVM** הנפוצה. היא מחלקת את הקוד שלנו ל-3 שכבות:

1. **Model** – הנתונים והלוגיקה העסקית (למשל אצלנו בקוד: `InventoryRepository`, `InventoryItem`).
 2. **ViewModel** – מתווך בין ה-View ל-Model. אחראי על עיבוד נתונים, ניהול מצבים, ולא מכיר את ה-UI. (למשל אצלנו בקוד: `InventoryViewModel`).
 3. **View (Fragment/Activity)** – מציג את הנתונים למשתמש ומאזין לשינויים מה-ViewModel (דרך `LiveData`). (למשל אצלנו בקוד: `InventoryFragment`).
- בעקבות החלוקה הנ"ל, כל תהליך עובר זרימה מסודרת בין packages שונים בפרויקט. ניתן לדוגמה את תהליך **הוספת פריט למלאי**. אנו נתמקד בתהליך זה לאורך המטלה.

חלק א' - Test Scenario Planning

Test Scenario 1: Valid input for adding a new inventory item (without image)

Scenario Name: Add inventory item (valid input)

Preconditions:

1. The app is installed and running.
2. The user is signed in (Firebase user exists).
3. The user is on the **Inventory screen** (`InventoryFragment`).
4. The inventory listener is active (the list is loaded and displayed).

Steps:

1. Click the **“+” FloatingActionButton** (Add Item).
2. In the dialog (`AddItemDialogFragment`), enter item name – **“מיטה”**.
3. Enter description – **“כבד, לפרק לפני ההובלה”**.
4. Choose room type from spinner – **“חדר שינה”**.
5. Check **Fragile** checkbox – **unchecked** (because a bed isn't fragile).
6. Enter quantity – **“1”**.

7. Do **not** add an image.
8. Click the **“Save”** button.

Expected Result:

1. The dialog closes automatically.
2. A toast message appears: **“נוסף בהצלחה”**.
3. The inventory list updates (via listener) and shows the new item card:
 - Name: **“מיטה”**
 - Room: **“חדר: חדר שינה”**
 - Quantity: **“x1”**
 - Date: visible as **“נוסף ב: dd/MM/yyyy”**
4. The new record is saved in the database for the current user (**ownerId = uid**).

Test Scenario 2: Adding an item with empty required field (invalid input test)

Scenario Name: Add inventory item with empty name (invalid input)

Preconditions:

1. The app is installed and running.
2. The user is signed in.
3. The user is on the **Inventory screen** (**InventoryFragment**).

Steps:

1. Click the **“+” FloatingActionButton** (Add Item).
2. Leave item name **empty**.
3. Enter description – **“קרטון עם כלי מטבח”**.
4. Choose room type – **“מטבח”**.
5. Enter quantity – **“3”**.
6. Click **“Save”**.

Expected Result:

1. The app stays in the Add Item dialog (does not close).
2. A toast message appears: **“נא להזין שם פריט”**.

3. No new item is added to the inventory list.
4. No new record is saved in the database.

Test Scenario 3: Deleting an existing inventory item record

Scenario Name: Delete inventory item

Preconditions:

1. The app is installed and running.
2. The user is signed in.
3. The user is on the **Inventory screen** (`InventoryFragment`).
4. At least one inventory item exists in the list (e.g., "ארון", room "סלון", quantity 1).

Steps:

1. Locate the item card "ארון" in the inventory list.
2. Click the **delete (trash) icon** (`btnDeleteItem`).
3. A confirmation dialog appears with the text: "למחוק את ארון?"
4. Click "כן".

Expected Result:

1. The item is deleted from the database (by `item.getId()`).
2. A toast message appears: "נמחק".
3. The item disappears from the list immediately (list refreshes via listener).
4. If the list becomes empty, the empty text message (`textEmptyInventory`) becomes visible.

חלק ב' - Unit Tests (Mocks)

בקובץ הפרויקט צורף הקובץ `InventoryViewModelUnitTests.java` – זהו הקובץ המכיל את הטסטים.

1. המשתמש ממלא פרטי פריט (שם, חדר, כמות) ולוחץ "הוסף" בדיאלוג `(AddItemDialogFragment)`.
 2. הפרגמנט קורא לפונקציה `viewModel.addItem(...)`.
 3. ה-`ViewModel` בודק שיש משתמש מחובר `(getCurrentUserId)` ומעביר את הנתונים ל-`InventoryRepository`.
 4. ה-`Repository` מדבר עם `Firebase` ושולח את הפריט ל-`Firestore` (ואת התמונה ל-`Storage` אם יש).
 5. כשהפעולה מסתיימת, ה-`ViewModel` מעדכן את ה-`LiveData: addSuccess` ו-`toastMessage`.
 6. ה-`Fragment` מאזין ל-`LiveData` ומציג למשתמש הודעת `Toast` ("נוסף בהצלחה") או סוגר את הדיאלוג.
- כלומר מה-`Fragment` (ה-`View` בעצם) עוברים ל-`ViewModel` ולאחר מכן ל-`Repository` (שזה ה-`Model`). לאחר הבנה בסיסית ניתן לעבור לטסטים עצמם. מכיוון שבחרנו בתהליך "הוספת פריט", החלטנו על **Unit 3 Tests** רלוונטיים במחלקה `InventoryViewModelUnitTests`:

testAddItem_Success

- בודק את מתודת `addItem` בתרחיש של "נתיב שמח" (Happy Path): יש משתמש מחובר, הנתונים תקינים, והשרת מחזיר תשובה חיובית.
- מתעסק בלוודא שה-`ViewModel` מעדכן את ה-`LiveData` של `addSuccess` ל-`true`, שההודעה למשתמש היא "נוסף בהצלחה", ושהקריאה לרפוזיטורי בוצעה בדיוק פעם אחת.

testAddItem_Failure

- בודק את מתודת `addItem` בתרחיש של כשל טכני (למשל: שגיאת רשת ב-`Firebase` או זריקת `Exception` מהשרת).
- מתעסק בלוודא שה-`ViewModel` "תופס" את השגיאה, מעדכן את ה-`toastMessage` עם תיאור השגיאה המתאים, ומונע קריסה של האפליקציה.

testAddItem_NoUserLoggedIn

- בודק את הלוגיקה העסקית והוולידציה לפני ביצוע הפעולה: ניסיון להוסיף פריט כאשר `getCurrentUserId` מחזיר `null` (אין משתמש מחובר).
- מתעסק בלוודא שה-`ViewModel` עוצר את הפעולה מיד ומוודא באמצעות `verify(..., never())` ששום קריאה לא נשלחה ל-`Repository` ולשרת, ובכך מגן על המסד נתונים מפני מידע שגוי.

כל טסט משתמש במוקים (Mocks) של `InventoryRepository` (באמצעות ספריית Mockito) כדי לחקות את התנהגות ה-Firebase ולבדוק את הלוגיקה העסקית של ה-`ViewModel` במנותק מהרשת. הם כוללים סנכרון של פעולות אסינכרוניות (Tasks) עם `.idle()` (`Looper.getMainLooper().shadowOf()`) כדי לוודא שה-`LiveData` מתעדכן לפני שורות ה-Assertion (הבדיקות).

חלק ג' - User Interface Tests (Espresso)

המשתמש מפעיל את האפליקציה, מבצע התחברות (במידת הצורך), מנווט למסך "המעבר שלי" ומשם למסך המלאי (Inventory). הטסט מחקה משתמש אנושי: הוא מקליד טקסט בשדות הקלט (EditText), לוחץ על כפתורים (Button, FAB), גולל ברשימה (RecyclerView) ומאמת שאלמנטים ויזואליים מופיעים על המסך כצפוי. בניגוד ל-Unit Tests, כאן אנו בודקים את האינטגרציה המלאה בין ה-View (ה-XML והפרגמנט) לבין הלוגיקה וה-Firebase בזמן אמת. לאחר הבנה בסיסית של זרימת המסכים, החלטנו על UI Tests 2 רלוונטיים במחלקה `InventoryUITest`:

1. `testAddItemToInventory` בודק את תהליך הוספת פריט חדש למלאי מקצה לקצה. מתעסק בביצוע התחברות אוטומטית (אם מסך הלוגין מוצג), ניווט למסך המלאי, לחיצה על כפתור ההוספה (+), מילוי פרטי הפריט (שם ייחודי, תיאור, כמות) ולחיצה על "שמור". הבדיקה מסתיימת באימות (Assertion) שהפריט החדש עם השם הייחודי אכן התווסף ומוצג בתוך ה-`RecyclerView`.
2. `testViewItemDetails` בודק את הצגת הפרטים המלאים של פריט ספציפי מתוך הרשימה. מתעסק ביצירת פריט עם תיאור ייחודי (כדי למנוע בלבול עם פריטים ישנים במסד הנתונים), איתור הפריט הספציפי ברשימה, ביצוע לחיצה על כפתור "הצג פירוט מלא" (שנמצא בתוך ה-`ViewHolder` של הפריט), ואימות שדיאלוג הפרטים נפתח ומכיל את הטקסט (התיאור) שהזנו.

היבטים טכניים: הטסטים משתמשים ב-`GrantPermissionRule` כדי לאשר אוטומטית הרשאות מערכת קריטיות (כמו מיקום והתראות) שעלולות לחסום את ה-UI ולכשיל את הטסט. בנוסף, השתמשנו ב-`System.currentTimeMillis()` ליצירת נתונים ייחודיים בכל ריצה כדי למנוע שגיאות של `AmbiguousViewMatcherException` (כפילות נתונים), ופתרנו התנגשויות גרסאות בספריית `protobuf` ב-`build.gradle` כדי לאפשר עבודה מול Firebase בסביבת הטסטים.

חלק ד' - Code Review in Pairs

שם הקובץ: CoachHomeActivity

שם המגישה: עמית ביטון 213763519

סקירה כללית:

המחלקה CoachHomeActivity אחראית על מסך הבית של מאמן בתוך אפליקציית StepBuyStep. הקוד מנהל את תהליך הטעינה וההצגה של נתוני המאמן מתוך Firebase, כולל מזהה המאמן, שם המשתמש, מספר מתאמנים פעילים ומספר בקשות ממתכונות. בנוסף, המחלקה שולפת ומציגה רשימת אימונים קרובים ממסד הנתונים Firestore באמצעות RecyclerView ו-UpcomingWorkoutAdapter. הקוד כולל אינטראקציה עם רכיבי UI שונים, ניווט למסכים נוספים, ניהול פעולות משתמש כמו התנתקות והעתקת מזהה המאמן ללוח, וכן פתיחת דיאלוג לשליחת הודעה קבוצתית. הלוגיקה משלבת עבודה אסינכרונית מול Firestore, עיבוד נתונים והצגת מצב ממשק משתמש דינמי בהתאם לתוצאות.

3 הערות מרכזיות לשיפור:

1. ריבוי אחריות בתוך ה-Activity:

המחלקה CoachHomeActivity מבצעת מספר משימות שונות: שליפת נתונים מ-Firebase, עיבוד הנתונים (כגון מיפוי אימונים לאובייקטים), ועדכון רכיבי ה-UI. ריכוז כל הלוגיקה במחלקה אחת מקשה על קריאת הקוד ועל תחזוקתו. כאשר מתווספות דרישות חדשות, יש צורך לבצע שינויים במספר אזורים שונים בקובץ, דבר שעלול להוביל לשגיאות. מומלץ לפצל את הלוגיקה לפונקציות או מחלקות ייעודיות, כך שכל חלק יהיה אחראי על משימה ברורה ומוגדרת.

2. טיפול חלקי במצבי שגיאה:

במספר מקומות בקוד, במקרה של שגיאה בשליפת נתונים מ-Firebase, הקוד מפסיק את הפעולה ללא טיפול נוסף. לדוגמה, בפונקציות המשתמשות ב-addSnapshotListener, כאשר מתקבלת שגיאה, לא מתבצע רישום לוג ולא מוצגת הודעה למשתמש. בנוסף, בפונקציה fetchUpcomingWorkouts אין טיפול במקרה של כישלון השאילתה. מצב זה עלול להקשות על זיהוי תקלות ולגרום לממשק להישאר במצב לא ברור עבור המשתמש. מומלץ להוסיף טיפול מפורש במצבי שגיאה, כגון הצגת הודעה או רישום לוג.

3. חזרתיות בלוגיקה ועדכון UI:

בקוד קיימת חזרתיות בפעולות דומות, כגון בדיקות האם משתמש מחובר, קביעת ערכי ברירת מחדל לשדות אימון, ועדכון מצב התצוגה של רשימת האימונים (הצגה או הסתרה של רכיבים). חזרתיות זו מקשה על הקריאה ועל התחזוקה של הקוד, משום שכל שינוי עתידי יחייב עדכון במספר מקומות. ניתן לשפר זאת באמצעות יצירת פונקציות עזר שיטפלו בפעולות חוזרות ויאחדו את לוגיקת עדכון הממשק.

דוגמה לשיפור מבוסס משוב:

הפונקציה fetchUpcomingWorkouts משלבת שליפה של נתונים ממסד הנתונים, עיבוד הנתונים ועדכון רכיבי הממשק במקום אחד, דבר שמקשה על הקריאה וההבנה (כפי שציינתי בסעיף 1). ניתן לשפר את מבנה הקוד על ידי פיצול לפונקציות נפרדות:

```

private void fetchUpcomingWorkouts() {
    if (auth.getCurrentUser() == null) return;
    String uid = auth.getCurrentUser().getUid();

    db.collection("workouts")
        .whereEqualTo("coachId", uid)
        .orderBy("createdAt", Query.Direction.DESENDING)
        .limit(3)
        .get()
        .addOnSuccessListener(snapshot -> {
            List<UpcomingWorkoutAdapter.WorkoutItem> items =
                mapWorkoutItems(snapshot);
            updateUpcomingUI(items);
        });
}

private List<UpcomingWorkoutAdapter.WorkoutItem> mapWorkoutItems(
    QuerySnapshot snapshot) {
    List<UpcomingWorkoutAdapter.WorkoutItem> items = new ArrayList<>();
    for (QueryDocumentSnapshot doc : snapshot) {
        String type = doc.getString("type") != null ? doc.getString("type") :
"Workout";
        String date = doc.getString("date") != null ? doc.getString("date") :
"";
        String time = doc.getString("time") != null ? doc.getString("time") :
"";
        String location = doc.getString("location") != null ?
doc.getString("location") : "";
        items.add(new UpcomingWorkoutAdapter.WorkoutItem(
            doc.getId(), type, date, time, location, 0));
    }
    return items;
}

private void updateUpcomingUI(List<UpcomingWorkoutAdapter.WorkoutItem> items)
{
    if (items.isEmpty()) {
        rvUpcoming.setVisibility(View.GONE);
        cardNoUpcoming.setVisibility(View.VISIBLE);
        badgeUpcomingCount.setText("0");
    } else {
        rvUpcoming.setVisibility(View.VISIBLE);
        cardNoUpcoming.setVisibility(View.GONE);
        badgeUpcomingCount.setText(String.valueOf(items.size()));
        upcomingAdapter.setItems(items);
    }
}

```

השינוי משפר את קריאות הקוד, מפריד בין שלבים שונים של הלוגיקה, ומקל על תחזוקה וניפוי תקלות.

שם הקובץ: TraineeHomeScreen

שם המגישה: שירה בן ארצי 211520127

סקירה כללית:

המחלקה TrackingActivity אחראית על ניהול מסך מעקב פעילות של מתאמן באפליקציית StepBuyStep. הקוד בודק הרשאות נדרשות (מיקום, זיהוי פעילות ולעיתים גם התראות לפי גרסת Android), מבצע חיבור (bind) ל-TrackingService שמודד מרחק ומספר צעדים, ומעדכן את מסך הסטטיסטיקות בזמן אמת באמצעות Handler שמריץ משימה כל שנייה. בנוסף, המחלקה מאפשרת לבחור סוג פעילות (הליכה/ריצה) ולפי הבחירה שולפת מ-Firebase את הצידוד הטוב ביותר מתוך ה-inventory של המשתמש כדי לקבוע מכפיל מטבעות לקילומטר. בסיום אימון הקוד מחשב מטבעות לפי מרחק ומכפיל, מעדכן את יתרת המטבעות במסמך המשתמש, ושומר את פרטי האימון בהיסטוריית אימונים (training_history). המחלקה משלבת עבודה עם שירות רקע, UI דינמי, ושליפות/עדכונים אסינכרוניים מול Firebase.

3 הערות מרכזיות לשיפור:

1. שילוב של הרבה אחריות בתוך Activity אחת:

המחלקה TrackingActivity מרכזת בתוכה מספר תחומי אחריות שונים: ניהול הרשאות, ניהול חיבור לשירות (ServiceConnection), לוגיקת מעקב (start/stop), עדכון UI בזמן אמת, וחישוב תגמולים ועדכון מסד נתונים. ריכוז של כל החלקים יחד מקשה על קריאת הקוד ועל תחזוקה עתידית, במיוחד כאשר ירצו להוסיף יכולות כמו סוגי פעילויות נוספים, חישוב מורכב יותר של תגמולים או מסך סיכום אימון. מומלץ לפצל את הלוגיקה לפונקציות/מחלקות ייעודיות, למשל מחלקה שמטפלת בשליפת ציוד וחישוב מכפיל, ומחלקה שמטפלת בשמירת תוצאות אימון למסד הנתונים, כך שה-Activity תתמקד בניהול תצוגה ואירועי משתמש.

2. בעיות אפשריות ברצף ההתחלה והחיבור לשירות (Bug Detection):

ב-startSession מופעל startForegroundService/startService, ולאחר מכן מתבצעת בדיקה if return (trackingService == null || !isBound). בפועל, ייתכן מצב שבו המשתמש לוחץ Start לפני שהחיבור לשירות הושלם, ואז השירות כבר רץ ברקע אך startTracking() לא נקרא וה-UI לא נכנס למצב "אימון פעיל". זה עלול ליצור חוסר עקביות ולבלבל את המשתמש. בנוסף, ה-Runnable ממשיך לרוץ כל שנייה גם אם עדיין אין bind מלא, משום שהוא מתוזמן מחדש תמיד עם postDelayed, מה שיכול ליצור עומס מיותר. מומלץ להבטיח שהפעלת המעקב תקרה רק לאחר onStartServiceConnected, או לחסום את כפתור Start עד שהשירות מחובר, ולהתחיל את ה-handler רק אחרי חיבור מוצלח.

3. טיפול חסר במצבי כשלון ושלמות נתונים מול Firestore:

בפונקציה fetchBestEquipment מתבצעת שליפה מה-inventory והקוד בוחר פריט עם multiplier מקסימלי, אך אין addOnFailureListener. במקרה של כשלון (לדוגמה בעיית רשת או הרשאות), המשתמש לא יקבל חיווי והמסך עלול להציג מידע חלקי או לא מעודכן. בנוסף, הקוד מניח ש-name תמיד קיים כאשר נמצא פריט טוב יותר, אך doc.getString("name") יכול להחזיר null ואז יוצג "Equipment: null". גם ב-saveSessionToHistory מתבצע finish() גם במקרה של כשלון כתיבה, כך שהמשתמש לא יודע שההיסטוריה לא נשמרה. מומלץ להוסיף טיפול בכשלון של שליפות/כתיבות, ולהציג הודעה מתאימה או לוג לצורך ניפוי תקלות, וכן לבצע בדיקות null לשדות שנשלפים.

דוגמה לשיפור מבוסס משוב:

הפונקציה `startSession` מערבת הפעלה של שירות, בדיקת `bind`, התחלת מעקב ועדכון UI. כדי למנוע מצב שבו השירות רץ אך המעקב לא מתחיל בגלל `bind` שעדיין לא הושלם (כפי שציינתי בסעיף 2), ניתן להפריד את הלוגיקה כך שהתחלת המעקב תתבצע רק לאחר חיבור מוצלח לשירות:

```
private void startSession() {
    Intent intent = new Intent(this, TrackingService.class);
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        startForegroundService(intent);
    } else {
        startService(intent);
    }

    if (!isBound) {
        bindService();
        btnStart.setEnabled(false);
        return;
    }

    startTrackingFlow();
}

private void startTrackingFlow() {
    if (trackingService == null) return;

    trackingService.startTracking();
    btnStart.setVisibility(View.GONE);
    btnStop.setVisibility(View.VISIBLE);
    setRadioGroupEnabled(false);
    handler.post(updateStatsRunnable);
}

@Override
public void onServiceConnected(ComponentName name, IBinder service) {
    TrackingService.LocalBinder binder = (TrackingService.LocalBinder)
service;
    trackingService = binder.getService();
    isBound = true;

    btnStart.setEnabled(true);
}
```

השינוי משפר את יציבות זרימת העבודה, מונע מצבים שבהם השירות רץ ללא מעקב פעיל, ומייצר התנהגות ברורה יותר מבחינת UI ותזמון החיבור לשירות.

שם הקובץ: HistoryTraineeActivity

שם המגיש: עלאא אלדין אבו חגלה.

ת.ז. המגיש: 322231580.

סקירה כללית:

המחלקה HistoryTraineeActivity מציגה את היסטוריית האימונים של המתאמן באפליקציית StepBuyStep. הקוד אחראי על שליפת נתונים משני אוספים שונים ב-Firebase Firestore (training_history ו-workouts), ביצוע אגרגציה של הנתונים (ספירת אימונים, מרחק כולל, קלוריות וניקוד), והצגתם ברשימה (RecyclerView) ובכותרת המסך. המחלקה מטפלת גם בלוגיקה של סינון אימונים עתידיים/עובריים ובעדכון הממשק במצבי הצלחה או כישלון. בנוסף, היא מכילה פונקציות עזר לפורמט תאריכים וניהול תצוגת מצב "ריק" (Empty State).

3 הערות מרכזיות לשיפור:

1. יעילות וביצועים (Client-Side Filtering & Date Parsing):

בפונקציה fetchPastScheduledWorkouts, הקוד שולף את כל האימונים שבהם המשתמש רשום, ורק לאחר מכן מבצע סינון (Filtering) בקוד Java באמצעות הפונקציה isPastWorkout. אם למשתמש יש היסטוריה ארוכה או אימונים עתידיים רבים, מדובר בבזבוז משאבי רשת וזיכרון. מומלץ לבצע את הסינון ברמת השאילתה ל-Firebase. בנוסף, בתוך הפונקציה isPastWorkout, נוצר אובייקט SimpleDateFormat חדש בכל קריאה. מאחר ופונקציה זו נקראת בתוך לולאה, הדבר יוצר עומס מיותר על ה-Garbage Collector. מומלץ להגדיר את הפורמטר כקבוע סטטי או להשתמש ב-LocalDate (ב-API עדכני).

2. ערכים קבועים (Magic Numbers) וקריאות:

הלוגיקה לחישוב קלוריות וניקוד משתמשת במספרים "קסומים" ($dist * 60$ לקלוריות, $dist * 100$ לניקודות) ללא הסבר או שימוש בקבועים (Constants). הדבר פוגע בקריאות הקוד ומקשה על שינוי עתידי של הנוסחה. מומלץ להוציא ערכים אלו לקבועים בעלי שם משמעותי (למשל: CALORIES_PER_KM). כמו כן, ישנה חזרתיות רבה בבדיקות Null (למשל `if (type == null)`) בתוך הלולאות, מה שמאריך את הקוד ומסרבל אותו.

3. מבנה הקוד והפרדת אחריות (Single Responsibility):

הפונקציה fetchHistory מבצעת עבודה רבה מדי: היא גם שולפת נתונים, גם מעבדת אותם (מחשבת סכומים), גם יוצרת אובייקטים של HistoryItem וגם יוזמת את הקריאה הבאה בשרשרת (fetchPastScheduledWorkouts). מבנה ה-"Callback Hell" (קריאה אסינכרונית בתוך קריאה אסינכרונית) מקשה על הבנת זרימת הנתונים ועל טיפול בשגיאות. מומלץ לפצל את עיבוד הנתונים לפונקציות נפרדות ולהשתמש ב-Tasks.whenAllSuccess כדי לשלוף את הנתונים משני המקורות במקביל במקום בטור.

דוגמה לשיפור מבוסס משוב:

בפונקציה fetchHistory, הלוגיקה של שליפת הנתונים ועיבודם מעורבבת (כפי שציינתי בהערה 3). השיפור להלן מפריד את הלוגיקה: פונקציה אחת אחראית על השליפה, ופונקציה נפרדת אחראית על המיפוי (Mapping) והחישובים. זה הופך את הקוד לקריא יותר, מודולרי, וקל לבדיקה.

```

private void fetchHistory() {
    String uid = auth.getUid();
    if (uid == null) return;

    db.collection("training_history")
        .whereEqualTo("userId", uid)
        .orderBy("timestamp", Query.Direction.DESENDING)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            // העברת האחריות לעיבוד הנתונים לפונקציה נפרדת
            processHistoryData(queryDocumentSnapshots, uid);
        })
        .addOnFailureListener(e -> {
            // Log או Toast -טיפול שגיאה בסיסי (ניתן להרחיב ל
            cardEmpty.setVisibility(View.VISIBLE);
        });
}

// פונקציית עיבוד נתונים חדשה - מטפלת בלוגיקה בלבד
private void processHistoryData(QuerySnapshot snapshot, String uid) {
    List<HistoryItem> items = new ArrayList<>();
    double totalDist = 0;
    int totalCalories = 0;
    int totalPoints = 0;

    // Magic Numbers-קבועים לחישוב (פותר את בעיית ה
    final int CALORIES_PER_KM = 60;
    final int POINTS_PER_KM = 100;

    for (QueryDocumentSnapshot doc : snapshot) {
        // ניתן להוסיף במחלקה) Null-שימוש בפונקציית עזר לטיפול ב
        String type = doc.getString("type");
        Double dist = doc.getDouble("distance");
        Long steps = doc.getLong("steps");
        String date = doc.getString("date");
        Long timestamp = doc.getLong("timestamp");

        // ערכי ברירת מחדל
        double safeDist = (dist != null) ? dist : 0.0;
        long safeSteps = (steps != null) ? steps : 0L;
        String safeType = (type != null) ? type : "Workout";
        long safeTimestamp = (timestamp != null) ? timestamp : 0L;

        // חישובים
        totalDist += safeDist;
    }
}

```

```

totalCalories += (int) (safeDist * CALORIES_PER_KM);
totalPoints += (int) (safeDist * POINTS_PER_KM);

// יצירת פריט לרשימה
String subtitle = String.format(Locale.getDefault(), "%.2f km • %d steps", safeDist, safeSteps);
String iconType = safeType.toLowerCase().contains("run") ? "run" : "walk";

items.add(new HistoryItem(doc.getId(), safeType, subtitle, date, safeTimestamp, iconType));
}

// המשך השרשרת לפונקציה הבאה עם הנתונים המעובדים
fetchPastScheduledWorkouts(uid, items, snapshot.size(), totalDist, totalCalories, totalPoints);
}

```

הסבר על השיפור:

1. **קריאות (Readability):** הפונקציה `fetchHistory` כעת קצרה וברורה. קל להבין שהיא שולפת נתונים ואז קוראת ל-`processHistoryData`.
2. **תחזוקה (Maintainability):** הוספנו קבועים לחישוב הקלוריות והנקודות, כך ששינוי עתידי דורש עדכון במקום אחד בלבד.
3. **ניקיון (Clean Code):** הטיפול בערכי null מסודר יותר ומשתמש במשתנים זמניים ברורים `(safeDist, safeType)`.

שם הקובץ: LeaderBoardActivity

שם המגיש: דוד קיטינברג.

ת.ז. המגיש: 322315300.

סקירה כללית:

המחלקה LeaderBoardActivity מנהלת את מסך טבלת המובילים (Leaderboard) באפליקציה. הקוד אחראי על זיהוי תפקיד המשתמש (מאמן או מתאמן) והתאמת ממשק הניווט בהתאם. הלוגיקה המרכזית כוללת שליפה מורכבת מ-Firebase: תחילה איתור כל המתאמנים המשוויכים למאמן ספציפי, ולאחר מכן ביצוע שאילתות משנה לכל מתאמן כדי למשוך את הציוד (Inventory) שלו ולחשב את המכפיל (Multiplier) הגבוה ביותר. הנתונים מוצגים ב-RecyclerView ממיון לפי דירוג, והמחלקה כוללת גם אפשרות למאמנים להעניק מטבעות למתאמנים דרך דיאלוג ייעודי.

3 הערות מרכזיות לשיפור:

1. בעיית ביצועים ושליפות מרובות (The N+1 Query Problem): בפונקציה fetchTrainees, הקוד מבצע לולאה שיוצרת Task נפרד לכל מתאמן כדי לשלוף את הציוד שלו `((db.collection("users").document(userId).collection("inventory").document(itemId).get().getDocument()).getDocument())`. אם למאמן יש 20 מתאמנים, האפליקציה תבצע 21 קריאות ל-Firebase. זהו בזבז של משאבי רשת, סוללה ומכסת שאילתות. מומלץ לשקול דה-נורמליזציה של הנתונים (שמירת המכפיל הגבוה ביותר ישירות במסמך המשתמש) או צמצום השאילתות כדי למנוע עומס על המכשיר והשרת.

2. חזרתיות בקוד הממשק (UI Boilerplate & Redundancy): הפונקציות `resetNavItem` ו-`setNavItemActive` מכילות לוגיקה כמעט זהה שסורקת ידנית את עץ ה-Views של רכיבי הניווט (`getChildAt, instanceof`). גישה זו "שבירה" מאוד – שינוי קטן ב-XML של הניווט יגרום לקריסת הקוד או להפסקת העדכון הויזואלי. בנוסף, הקוד חוזר על עצמו עבור כל אחד מארבעת רכיבי הניווט. ניתן לשפר זאת משמעותית על ידי שימוש ב-View Binding או יצירת פונקציית עזר גנרית שמקבלת id ומבצעת את השינוי בצורה אחידה.

3. היעדר טיפול בשגיאות וקריסות פוטנציאליות: במספר נקודות קריטיות, כמו `Tasks.whenAllComplete`, הקוד מניח שהנתונים תמיד קיימים ותמיד תקינים. לדוגמה, הקריאה `(t.getResult().getDocuments().get(0).isEmpty())` עלולה לזרוק `IndexOutOfBoundsException` אם האוסף ריק, למרות הבדיקה של `isEmpty()`. כמו כן, אין `addOnFailureListener` מפורט לכל השלבים, מה שעלול להשאיר את המשתמש מול מסך טעינה אינסופי או "Empty State" ללא הסבר במקרה של תקלת תקשורת.

דוגמה לשיפור מבוסס משוב:

הפונקציה fetchTrainees כיום מאוד עמוסה ומערבבת בין שליפת רשימת המתאמנים לבין שליפת הציוד שלהם. נפריד את שלב עיבוד הנתונים והמיון לפונקציה נפרדת שתהיה אחראית על הפיכת ה-Snapshots לאובייקטים של המודל, מה שיספר את הקריאות והיכולת לתחזק את הלוגיקה של הדירוג:

```

private void fetchTrainees(Long coachId) { no usages
    if (coachId == null) return;

    db.collection("users")
        .whereEqualTo("role", "trainee")
        .whereEqualTo("coachID", coachId)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            processTraineesAndFetchStats(queryDocumentSnapshots);
        })
        .addOnFailureListener(e -> {
            Toast.makeText(this, "Failed to load trainees", Toast.LENGTH_SHORT).show();
            updateAdapter(new ArrayList<>());
        });
}

private void processTraineesAndFetchStats(QuerySnapshot traineeSnapshots) { 1 usage
    List<LeaderboardAdapter.LeaderboardItem> tempItems = new ArrayList<>();
    List<Task<QuerySnapshot>> inventoryTasks = new ArrayList<>();

    for (QueryDocumentSnapshot doc : traineeSnapshots) {
        String name = formatNameFromEmail(doc.getString("email"));
        LeaderboardAdapter.LeaderboardItem item = new LeaderboardAdapter.LeaderboardItem(doc.getId(), name, 0, 1, 1.0, "Unknown");
        tempItems.add(item);

        // יצירת חשימה לכל משתמש
        inventoryTasks.add(db.collection("users").document(doc.getId())
            .collection("inventory")
            .orderBy("multiplier", Query.Direction.DESENDING)
            .limit(1).get());
    }

    Tasks.whenAllComplete(inventoryTasks).addOnSuccessListener(results -> {
        finalizeLeaderboardData(tempItems, inventoryTasks);
    });
}
}

```

```

private void finalizeLeaderboardData(List<LeaderboardAdapter.LeaderboardItem> items, List<Task<QuerySnapshot>> tasks) {
    for (int i = 0; i < tasks.size(); i++) {
        Task<QuerySnapshot> t = tasks.get(i);
        if (t.isSuccessful() && !t.getResult().isEmpty()) {
            QueryDocumentSnapshot shoeDoc = (QueryDocumentSnapshot) t.getResult().getDocuments().get(0);
            Double m = shoeDoc.getDouble("multiplier");
            if (m != null) items.get(i).multiplier = m;
        }
    }

    // מיון ודירוג (הפרדת לוגיקה עסקית)
    Collections.sort(items, (o1, o2) -> Double.compare(o2.multiplier, o1.multiplier));
    for (int i = 0; i < items.size(); i++) {
        items.get(i).rank = i + 1;
    }
    updateAdapter(items);
}
}

```

הסבר על השיפור:

- **מודולריות:** פיצלנו את השאילתה הענקית לשלבים ברורים (fetch -> process -> finalize).
- **חסינות:** הוספנו טיפול ב-onFailure בסיסי כדי שהמשתמש ידע מה קורה.
- **ניקיון:** הוצאנו את לוגיקת המיון והעיבוד לפונקציה נפרדת, מה שמאפשר בעתיד לשנות את שיטת הניקוד מבלי לגעת בקוד השליפה מה-DB.